Finite State Machines in Verilog

Sequence detectors: design, always blocks, testbench, diagrams and waveforms

What is inside. The two questions you were given (**Q1**, **Q2**), four predicted questions (**P3**–**P6**) and a page of quick variations. Each question comes with the state meanings, a state diagram, a step-by-step picture of every transition, a trace table, the waveform, the RTL in short labelled slices, and a testbench that applies random input for 100 clocks.
Conventions used throughout. Active-high *asynchronous* reset to S_0. The testbench changes the input 1 ns after each rising edge, so each input value is held for one full clock and *sampled at the next rising edge*. In the waveforms the numbers on top are the rising edges, and edge k samples the input drawn just before it.

Contents

1 The 5-step recipe that solves every detector question **1**

2 Q2 (given) – non-overlapping "010" detector, 1-bit `serin` **3**
2.1 Building the state table step by step . . . 3
2.2 State diagram (Moore, 4 states) . . . 3
2.3 Step-by-step: watch the machine run on `101010010011` . . . 3
2.4 Trace table . . . 4
2.5 Waveform (Moore and Mealy versions on the same input) . . . 5
2.6 RTL – Moore version, slice by slice . . . 5
2.7 Mealy alternative (if asked, or to show you know both) . . . 6
2.8 Testbench – 100 random clocks, slice by slice . . . 7

3 Q1 (given) – "01" on a 2-bit `serin` for 3 consecutive clocks, overlap allowed **8**
3.1 Key idea: this is a counter, not a bit pattern . . . 8
3.2 State diagram (Moore) . . . 8
3.3 Step-by-step . . . 8
3.4 Trace table . . . 9
3.5 Waveform . . . 9
3.6 RTL, slice by slice . . . 10
3.7 Testbench, slice by slice . . . 10

4 Predicted P3 – overlapping "1011" detector (Mealy) **12**
4.1 Step-by-step . . . 12
4.2 Trace table and waveform . . . 13

5 Predicted P4 – "101": overlapping vs. non-overlapping (Moore) **15**
5.1 Waveform: same input, both machines . . . 15

6 Predicted P5 – "11" twice in a row on a 2-bit input (non-overlapping) **17**

7 Predicted P6 – serial number divisible by 3 **18**

8 Quick variations – state tables you can draw in 2 minutes **19**

1 The 5-step recipe that solves every detector question

1. **Name the states by "what I have seen so far".** For a pattern p of length n, the states are the prefixes of p: S_0 = nothing, S_1 = first bit, ... A **Mealy** machine needs n states. A **Moore** machine needs $n+1$, because the extra state "whole pattern seen" drives the output. For "value V on a multi-bit input for N consecutive clocks", the states simply count the run $(0, 1, \ldots, N$ in a row).

2. **The correct-bit arrow.** From each state, the bit that extends the prefix moves one state forward.
3. **The wrong-bit arrow.** Take (what you had + the wrong bit) and find its *longest ending that is still a start of the pattern*. Go to the state for that ending, or to S_0 if there is none.
4. **After a detection: overlap or not?**
   - *Overlapping*: treat the detected pattern like any other prefix, using the step-3 rule ("1011" → keep the trailing "1").
   - *Non-overlapping*: the detected bits are used up, so restart as if from S_0 with the new bit.
5. **Write three always blocks**: state register (`<=`), next-state logic (`always @(*)`, `=`, default first), output logic (Moore: from the state only; Mealy: from the state *and* the input).

	Moore	Mealy
Output depends on	present state only	present state *and* present input
States for n-bit pattern	$n+1$	n
When `det_out` rises	one clock *after* the last bit is sampled	in the *same* clock as the last bit (before the edge)
Glitches	none (output is a function of flip-flops)	can follow input glitches
Where output is written	in the bubble: `S3/1`	on the arrow: `in/out`

Marks are usually lost on

- a missing `default` or missing default assignment in `always @(*)` (this infers a latch, so the design is not synthesizable as intended);
- using `<=` in combinational logic or `=` in the clocked block;
- driving the testbench input exactly on `posedge clk` (a race). Use `@(posedge clk); #1;` or `@(negedge clk);`
- forgetting the self-loops: every state needs an arrow for *every* input value;
- non-overlap vs. overlap: only the arrows *leaving the final state* change.

Generic template (memorise this skeleton)

Three-block FSM template `template`

```
module my_fsm (input wire clk, rst, input wire serin, output reg det_out);
    localparam [1:0] S0 = 2'd0, S1 = 2'd1, S2 = 2'd2, S3 = 2'd3;
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst)          // 1. state register
        if (rst) state <= S0; else state <= next_state;

    always @(*) begin                             // 2. next-state logic
        next_state = S0;                          //    default first!
        case (state)
            S0: next_state = serin ? S0 : S1;     //    one line per bubble
            // ...
            default: next_state = S0;
        endcase
    end

    always @(*) det_out = (state == S3);          // 3. output (Moore)
endmodule
```

2 Q2 (given) – non-overlapping "010" detector, 1-bit `serin`

Question

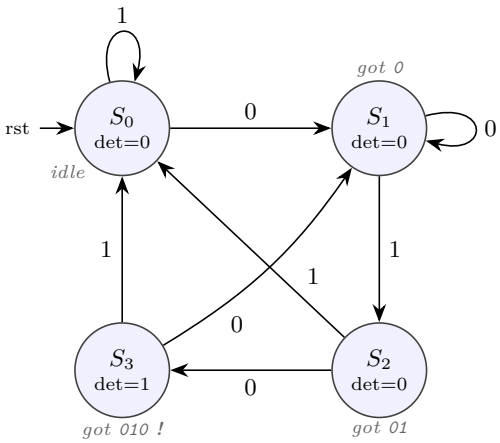
Model a non-overlapping sequence detector FSM that detects a sequence "010" in a single-bit serial input `serin`. The FSM asserts an output `det_out` on detecting a correct sequence. Verify the FSM with random inputs applied for an interval of 100 clocks.

2.1 Building the state table step by step

State	Meaning	`serin`=0	`serin`=1	Why
S_0	idle	S_1	S_0	0 starts the pattern; "1" is useless
S_1	got `0`	S_1	S_2	"00" still ends in "0" ⇒ stay
S_2	got `01`	S_3	S_0	"011" has no ending that starts "010"
S_3	got `010`, **det_out=1**	S_1	S_0	**non-overlap**: start again from scratch

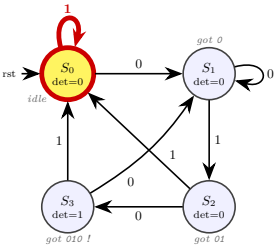
If the question said *overlapping*, only one arrow changes: $S_3 \xrightarrow{1} S_2$ (the last "0" of "010" is reused as the first "0" of "01").

2.2 State diagram (Moore, 4 states)



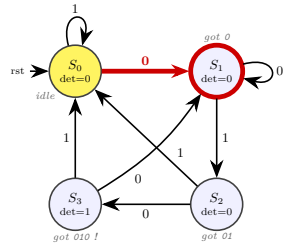
2.3 Step-by-step: watch the machine run on `101010010011`

Legend for the step-by-step pictures: present state next state transition taken on this clock edge



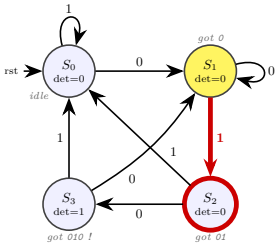
Clock 1: `serin=1`

$S_0 \to S_0$ | det_out=0 (after edge)



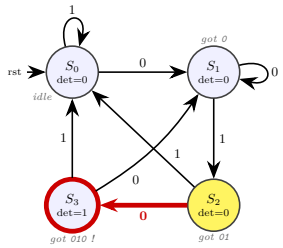
Clock 2: `serin=0`

$S_0 \to S_1$ | det_out=0 (after edge)



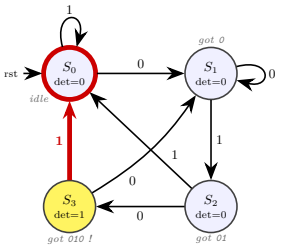
Clock 3: `serin=1`

$S_1 \to S_2$ | det_out=0 (after edge)



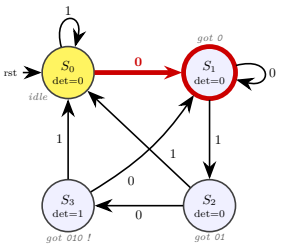
Clock 4: `serin=0`

$S_2 \to S_3$ | **det_out=1** (after edge)



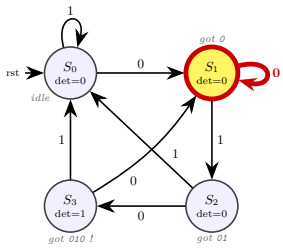
Clock 5: `serin=1`

$S_3 \to S_0$ | det_out=0 (after edge)



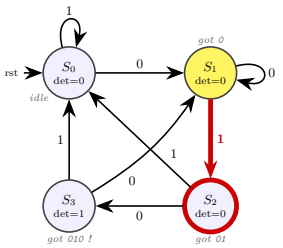
Clock 6: `serin=0`

$S_0 \to S_1$ | det_out=0 (after edge)



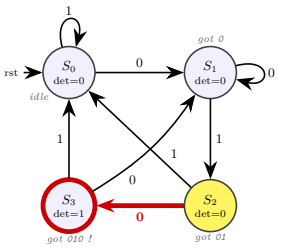
Clock 7: `serin=0`

$S_1 \to S_1$ | det_out=0 (after edge)



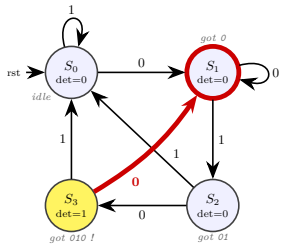
Clock 8: `serin=1`

$S_1 \to S_2$ | det_out=0 (after edge)



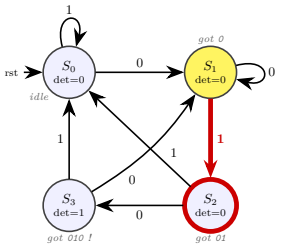
Clock 9: `serin=0`

$S_2 \to S_3$ | **det_out=1** (after edge)



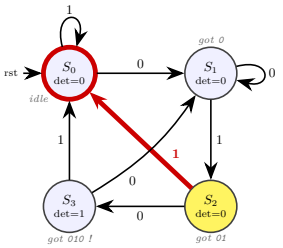
Clock 10: `serin=0`

$S_3 \to S_1$ | det_out=0 (after edge)



Clock 11: `serin=1`

$S_1 \to S_2$ | det_out=0 (after edge)



Clock 12: `serin=1`

$S_2 \to S_0$ | det_out=0 (after edge)

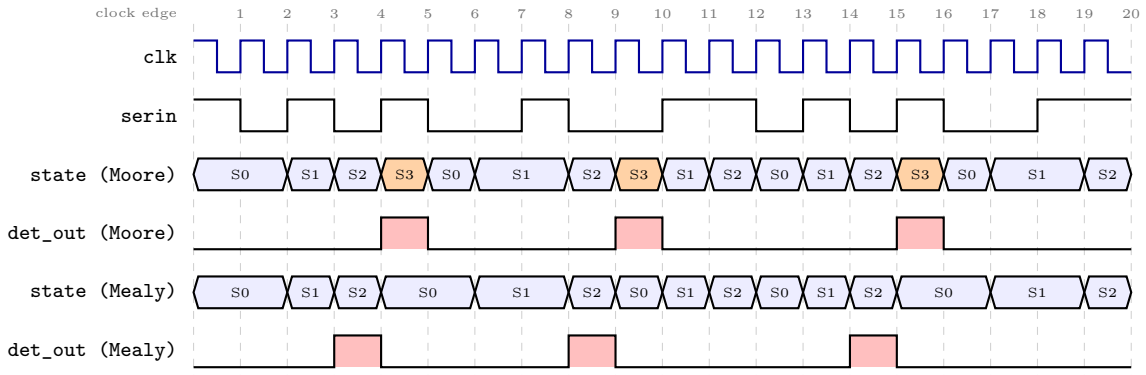
Notice the non-overlap at clocks 2–6

The input `01010` (clocks 2–6) contains "010" at clocks 2–4 *and* "010" at clocks 4–6, sharing the 0 of clock 4. Non-overlapping counts only the first one. At clock 5 the 1 sends $S_3 \to S_0$, so the shared 0 is not reused, and the next detection needs a brand-new "010".

2.4 Trace table

Clock edge	`serin` sampled	Present state	Next state	`det_out` after edge (Moore)
1	1	S_0	S_0	0
2	0	S_0	S_1	0
3	1	S_1	S_2	0
4	0	S_2	S_3	**1**
5	1	S_3	S_0	0
6	0	S_0	S_1	0
7	0	S_1	S_1	0
8	1	S_1	S_2	0
9	0	S_2	S_3	**1**
10	0	S_3	S_1	0
11	1	S_1	S_2	0
12	1	S_2	S_0	0
13	0	S_0	S_1	0
14	1	S_1	S_2	0
15	0	S_2	S_3	**1**
16	1	S_3	S_0	0
17	0	S_0	S_1	0
18	0	S_1	S_1	0
19	1	S_1	S_2	0
20	1	S_2	S_0	0

2.5 Waveform (Moore and Mealy versions on the same input)



Moore raises `det_out` for the clock *after* the edge that sampled the final 0. Mealy raises it while that final 0 is still on the input, one clock earlier and with one state fewer.

2.6 RTL – Moore version, slice by slice

Slice 1 – ports and state encoding `seq010_moore.v`

One state per "how much of 010 have I seen". Two bits are enough for four states.

```
`timescale 1ns/1ps
// Non-overlapping "010" sequence detector -- Moore FSM
module seq010_moore (
    input  wire clk,
    input  wire rst,       // active-high asynchronous reset
    input  wire serin,     // serial input, one bit per clock
    output reg  det_out    // high for one clock after "010" is seen
);
    localparam [1:0] S0 = 2'b00,   // idle (nothing useful yet)
                     S1 = 2'b01,   // got "0"
                     S2 = 2'b10,   // got "01"
                     S3 = 2'b11;   // got "010"  -> det_out = 1

    reg [1:0] state, next_state;
```

Slice 2 – state register (the only clocked block) `seq010_moore.v`

Non-blocking <=, reset to the idle state.

```
    // (1) state register
    always @(posedge clk or posedge rst) begin
        if (rst) state <= S0;
        else     state <= next_state;
    end
```

Slice 3 – next-state logic (combinational) `seq010_moore.v`

Blocking =, a default assignment so no latch is inferred, and one line per bubble of the diagram.

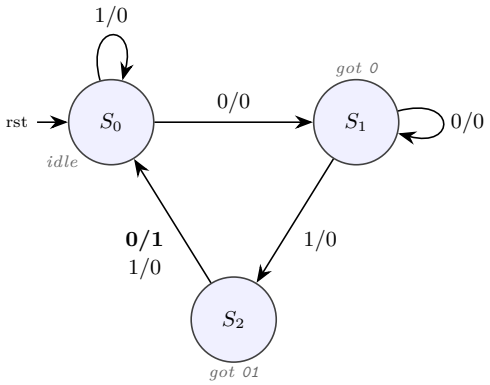
```
    // (2) next-state logic
    always @(*) begin
        next_state = S0;                          // default: no latch
        case (state)
            S0: next_state = (serin == 1'b0) ? S1 : S0;
            S1: next_state = (serin == 1'b1) ? S2 : S1;
            S2: next_state = (serin == 1'b0) ? S3 : S0;
            // NON-overlap: the "0" that completed 010 is NOT reused.
            // A new 0 starts a fresh sequence, a 1 goes back to idle.
            S3: next_state = (serin == 1'b0) ? S1 : S0;
            default: next_state = S0;
        endcase
    end
```

Slice 4 – output logic (Moore: state only) `seq010_moore.v`

The output depends on the state alone, so it is glitch-free and appears one clock after the last bit.

```
    // (3) output logic
    always @(*) begin
        det_out = (state == S3);
    end
endmodule
```

2.7 Mealy alternative (if asked, or to show you know both)



Mealy version – 3 states, output on the transition `seq010_mealy.v`

Same detector with one state fewer; `det_out` *rises in the same clock as the final 0.*

```
`timescale 1ns/1ps
// Non-overlapping "010" sequence detector -- Mealy FSM
module seq010_mealy (
    input  wire clk,
    input  wire rst,
    input  wire serin,
    output reg  det_out
);
    localparam [1:0] S0 = 2'd0,    // idle
                     S1 = 2'd1,    // got "0"
                     S2 = 2'd2;    // got "01"
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst)
        if (rst) state <= S0;
        else     state <= next_state;

    // next state AND output in one combinational block
    always @(*) begin
        next_state = S0;
        det_out    = 1'b0;
        case (state)
            S0: next_state = serin ? S0 : S1;
            S1: next_state = serin ? S2 : S1;
            S2: begin
                    next_state = S0;        // 011 -> idle, 010 -> idle (non-overlap)
                    det_out    = ~serin;    // "01" followed by 0: detected NOW
                end
            default: next_state = S0;
        endcase
    end
endmodule
```

2.8 Testbench – 100 random clocks, slice by slice

TB slice 1 – signals, DUT and clock tb_seq010.v

The testbench has no ports. Inputs are `reg`, outputs are `wire`.

```
`timescale 1ns/1ps
module tb_seq010;
    reg  clk = 1'b0, rst = 1'b1, serin = 1'b0;
    wire det_out;
    integer i, hits = 0, errors = 0;
    reg [1:0] hist;  integer n;  reg exp;      // golden model (slice 4)

    seq010_moore dut (.clk(clk), .rst(rst), .serin(serin), .det_out(det_out));

    always #5 clk = ~clk;                      // 10 ns clock period
```

TB slice 2 – reset, then 100 clocks of random input tb_seq010.v

Change the input 1 ns after the rising edge so it never races the clock.

```
    initial begin
        $dumpfile("tb_seq010.vcd");            // open in GTKWave
        $dumpvars(0, tb_seq010);
        repeat (2) @(posedge clk);             // hold reset for 2 clocks
        #1 rst = 1'b0;
        for (i = 0; i < 100; i = i + 1) begin
            serin = $random;                   // LSB of a random number: 0/1
            @(posedge clk); #1;                // held for exactly one clock
        end
        $display("DONE: %0d detections, %0d errors", hits, errors);
        $finish;
    end
```

TB slice 3 – print what happens on every clock tb_seq010.v

`$strobe` prints at the end of the time step, so it shows the new state.

```
    always @(posedge clk)
        if (!rst)
            $strobe("%6t  serin=%b  ->  state=S%0d  det_out=%b",
                    $time, serin, dut.state, det_out);
```

TB slice 4 – self-check against a golden model (bonus marks) tb_seq010.v

Keep the last 2 bits and count fresh bits since the last hit. The non-overlap rule is that a hit needs 3 fresh bits.

```
    always @(posedge clk or posedge rst)
        if (rst) begin
            hist <= 2'b00;  n <= 0;  exp <= 1'b0;
        end else begin
            hist <= {hist[0], serin};
            if (n >= 2 && {hist, serin} == 3'b010) begin
                exp <= 1'b1;  n <= 0;              // hit: restart counting
            end else begin
                exp <= 1'b0;  n <= (n < 2) ? n + 1 : n;
            end
        end

    always @(negedge clk)                      // compare mid-cycle
        if (!rst) begin
            if (det_out) hits = hits + 1;
            if (det_out !== exp) begin
                errors = errors + 1;
                $display("MISMATCH at %0t: det_out=%b expected=%b",
                         $time, det_out, exp);
            end
        end
endmodule
```

3 Q1 (given) – "01" on a 2-bit `serin` for 3 consecutive clocks, overlap allowed

Question

Write a synthesizable behavioral model of an FSM that asserts an output `det_out` if the value "01" is received in a 2-bit input `serin` for three consecutive clocks with overlaps allowed. Write a test bench for verifying the FSM with random inputs applied for 100 clocks.

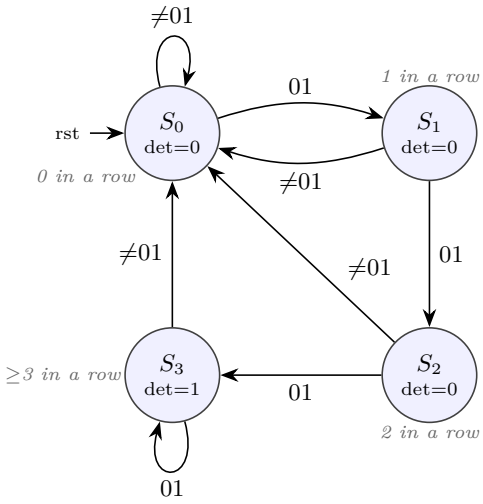
3.1 Key idea: this is a counter, not a bit pattern

Here `serin` is a 2-bit *bus*, and "01" is one value of it (`2'b01`). The FSM only has to count how many clocks *in a row* it has seen `01`. Any other value (00, 10, 11) breaks the run. *Overlap allowed* means a 4th consecutive 01 is a new run of three (clocks 2–4), so `det_out` stays high, which is the self-loop on S_3.

State	Meaning	`serin`=01	`serin`≠01
S_0	0 in a row	S_1	S_0
S_1	1 in a row	S_2	S_0
S_2	2 in a row	S_3	S_0
S_3	≥3 in a row, **det_out=1**	S_3 (overlap)	S_0

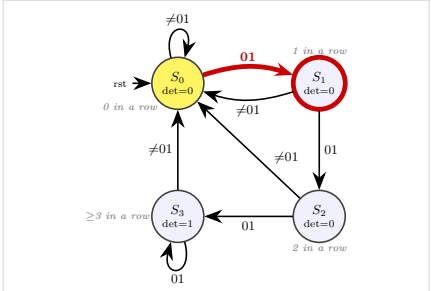
Non-overlapping variant: $S_3 \xrightarrow{01} S_1$ (the run restarts at 1).

3.2 State diagram (Moore)



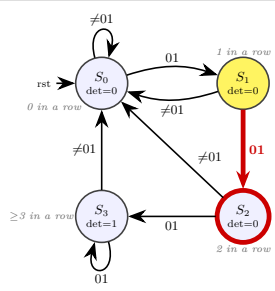
3.3 Step-by-step

Legend for the step-by-step pictures: present state next state transition taken on this clock edge



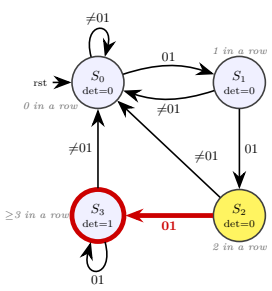
Clock 1: `serin=01`

$S_0 \rightarrow S_1$ | det_out=0 (after edge)



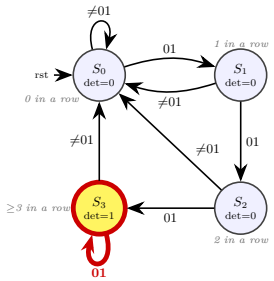
Clock 2: `serin=01`

$S_1 \rightarrow S_2$ | det_out=0 (after edge)



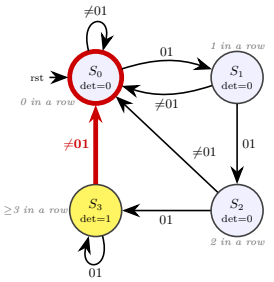
Clock 3: `serin=01`

$S_2 \rightarrow S_3$ | **det_out=1** (after edge)



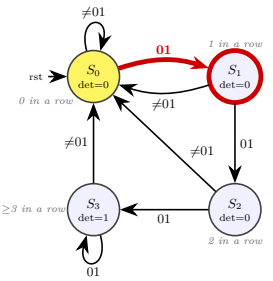
Clock 4: `serin=01`

$S_3 \rightarrow S_3$ | **det_out=1** (after edge)



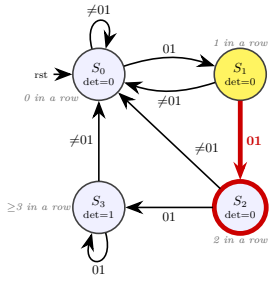
Clock 5: `serin=10`

$S_3 \rightarrow S_0$ | det_out=0 (after edge)



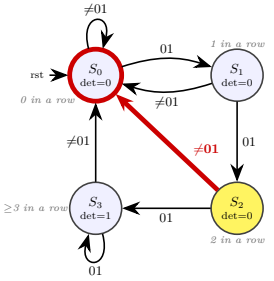
Clock 6: `serin=01`

$S_0 \rightarrow S_1$ | det_out=0 (after edge)



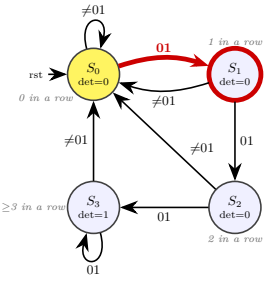
Clock 7: `serin=01`

$S_1 \rightarrow S_2$ | det_out=0 (after edge)



Clock 8: `serin=00`

$S_2 \rightarrow S_0$ | det_out=0 (after edge)



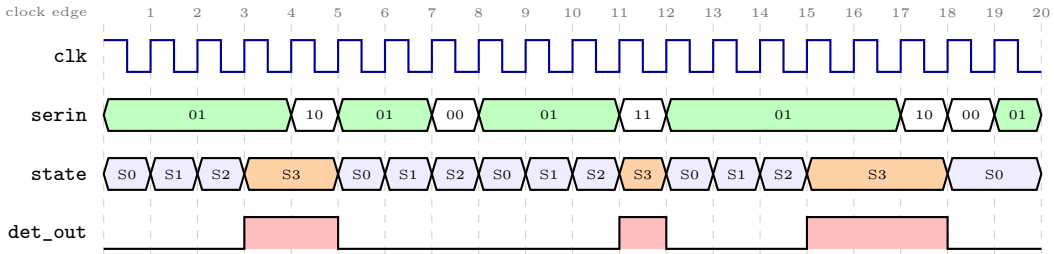
Clock 9: `serin=01`

$S_0 \rightarrow S_1$ | det_out=0 (after edge)

3.4 Trace table

Clock edge	`serin` sampled	Present state	Next state	`det_out` after edge (Moore)
1	01	S_0	S_1	0
2	01	S_1	S_2	0
3	01	S_2	S_3	**1**
4	01	S_3	S_3	**1**
5	10	S_3	S_0	0
6	01	S_0	S_1	0
7	01	S_1	S_2	0
8	00	S_2	S_0	0
9	01	S_0	S_1	0
10	01	S_1	S_2	0
11	01	S_2	S_3	**1**
12	11	S_3	S_0	0
13	01	S_0	S_1	0
14	01	S_1	S_2	0
15	01	S_2	S_3	**1**
16	01	S_3	S_3	**1**
17	01	S_3	S_3	**1**
18	10	S_3	S_0	0
19	00	S_0	S_0	0
20	01	S_0	S_1	0

3.5 Waveform



Green = 01. `det_out` goes high one clock after the 3rd consecutive 01 is sampled and *stays* high while the run continues (overlap). A single non-01 value drops it.

3.6 RTL, slice by slice

Slice 1 – ports: serin is now 2 bits wide det01x3.v

Each state counts how many clocks in a row the value 01 has been seen.

```
`timescale 1ns/1ps
// Asserts det_out when serin == 2'b01 for 3 consecutive clocks.
// Overlap allowed: a 4th, 5th ... consecutive 01 keeps det_out high.
module det01x3 (
    input  wire       clk,
    input  wire       rst,        // active-high asynchronous reset
    input  wire [1:0] serin,      // 2-bit input every clock
    output reg        det_out
);
    localparam [1:0] S0 = 2'd0,   // 0 consecutive "01"
                     S1 = 2'd1,   // 1 consecutive "01"
                     S2 = 2'd2,   // 2 consecutive "01"
                     S3 = 2'd3;   // >= 3 consecutive  -> det_out = 1

    reg [1:0] state, next_state;
```

Slice 2 – state register det01x3.v

Identical in every FSM you will write.

```
    always @(posedge clk or posedge rst) begin
        if (rst) state <= S0;
        else     state <= next_state;
    end
```

Slice 3 – next-state logic det01x3.v

Any value other than 01 breaks the run, so the default is S0. The S3→S3 self-loop is the overlap.

```
    always @(*) begin
        next_state = S0;                  // anything except 01 -> back to S0
        if (serin == 2'b01) begin
            case (state)
                S0: next_state = S1;
                S1: next_state = S2;
                S2: next_state = S3;
                S3: next_state = S3;      // OVERLAP: stay, keep detecting
                                          // (non-overlap would be: S1)
                default: next_state = S0;
            endcase
        end
    end
```

Slice 4 – Moore output det01x3.v

```
    always @(*) begin
        det_out = (state == S3);
    end
endmodule
```

3.7 Testbench, slice by slice

TB slice 1 – signals, DUT, clock tb_det01x3.v

```
`timescale 1ns/1ps
module tb_det01x3;
    reg        clk = 1'b0, rst = 1'b1;
    reg  [1:0] serin = 2'b00;
    wire       det_out;
    integer    i, hits = 0, errors = 0, run;
    reg        exp;

```

```
    det01x3 dut (.clk(clk), .rst(rst), .serin(serin), .det_out(det_out));

    always #5 clk = ~clk;
```

TB slice 2 – 100 random clocks tb_det01x3.v

Pure random hits 01 only 1 time in 4, so three in a row is rare. The commented line biases the input so you see detections.

```
    initial begin
        $dumpfile("tb_det01x3.vcd");
        $dumpvars(0, tb_det01x3);
        repeat (2) @(posedge clk);
        #1 rst = 1'b0;
        for (i = 0; i < 100; i = i + 1) begin
            serin = $random;                                  // 00/01/10/11
            // serin = ({$random} % 3 != 0) ? 2'b01 : $random;  // more hits
            @(posedge clk); #1;
        end
        $display("DONE: %0d clocks with det_out=1, %0d errors", hits, errors);
        $finish;
    end

    always @(posedge clk)
        if (!rst)
            $strobe("%6t  serin=%b  ->  state=S%0d  det_out=%b",
                    $time, serin, dut.state, det_out);
```

TB slice 3 – golden model: a run-length counter tb_det01x3.v

```
    always @(posedge clk or posedge rst)
        if (rst)                    begin run <= 0; exp <= 1'b0; end
        else if (serin == 2'b01)  begin run <= (run < 3) ? run + 1 : run;
                                        exp <= (run >= 2);              end
        else                        begin run <= 0; exp <= 1'b0; end

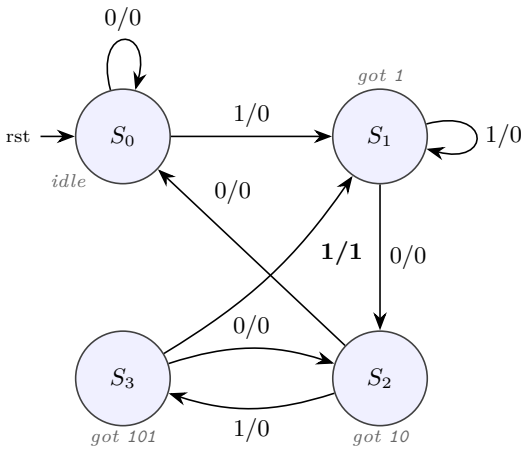
    always @(negedge clk)
        if (!rst) begin
            if (det_out) hits = hits + 1;
            if (det_out !== exp) begin
                errors = errors + 1;
                $display("MISMATCH at %0t", $time);
            end
        end
endmodule
```

4 Predicted P3 – overlapping “1011” detector (Mealy)

Likely wording

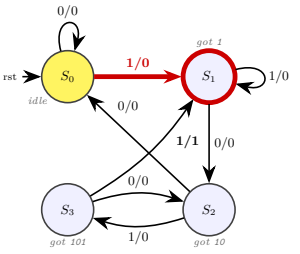
Design a Mealy FSM that detects the overlapping sequence “1011” on a serial input and asserts `det_out`. Draw the state diagram and write the testbench for 100 random clocks.

State	Seen	in=0	in=1	Reasoning for the “wrong” bit
S_0	–	$S_0/0$	$S_1/0$	
S_1	`1`	$S_2/0$	$S_1/0$	“11” ends in “1”
S_2	`10`	$S_0/0$	$S_3/0$	“100” has no useful ending
S_3	`101`	$S_2/0$	$S_1/\mathbf{1}$	“1010” ends in “10”; “1011” ends in “1” (overlap)



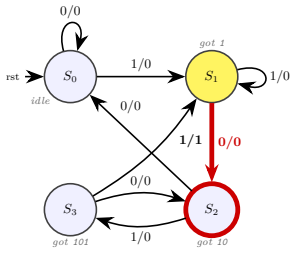
4.1 Step-by-step

Legend for the step-by-step pictures: present state next state transition taken on this clock edge



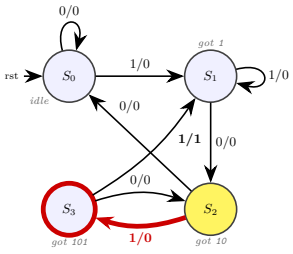
Clock 1: `serin=1`

$S_0 \rightarrow S_1$ | det_out=0 (before edge)



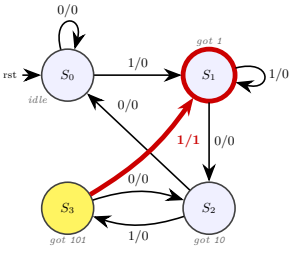
Clock 2: `serin=0`

$S_1 \rightarrow S_2$ | det_out=0 (before edge)



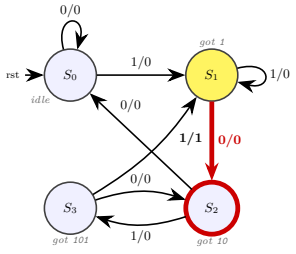
Clock 3: `serin=1`

$S_2 \rightarrow S_3$ | det_out=0 (before edge)



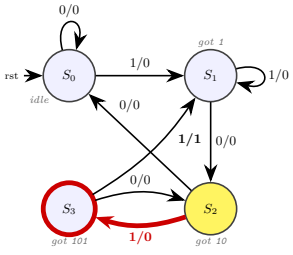
Clock 4: `serin=1`

$S_3 \rightarrow S_1$ | **det_out=1** (before edge)



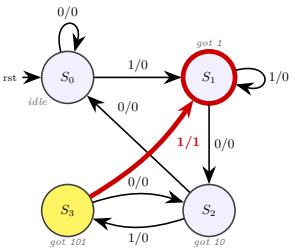
Clock 5: `serin=0`

$S_1 \rightarrow S_2$ | det_out=0 (before edge)



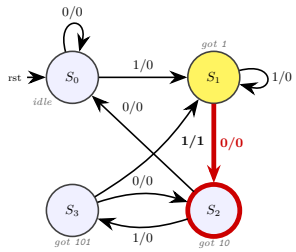
Clock 6: `serin=1`

$S_2 \rightarrow S_3$ | det_out=0 (before edge)



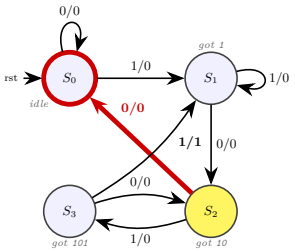
Clock 7: `serin=1`

$S_3 \rightarrow S_1$ | **det_out=1** (before edge)



Clock 8: `serin=0`

$S_1 \rightarrow S_2$ | det_out=0 (before edge)

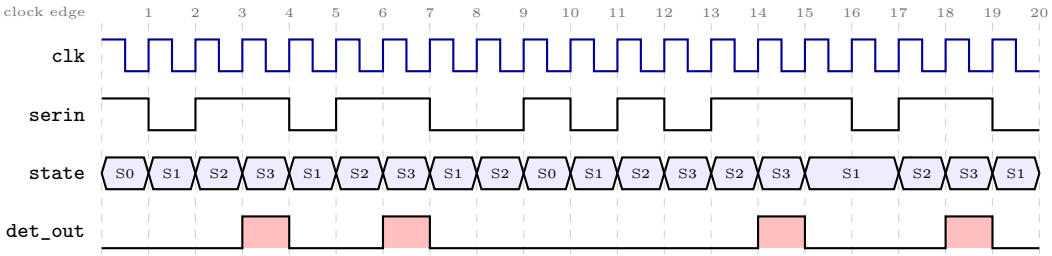


Clock 9: `serin=0`

$S_2 \rightarrow S_0$ | det_out=0 (before edge)

4.2 Trace table and waveform

Clock edge	`serin` sampled	Present state	Next state	`det_out` before edge (Mealy)
1	1	S_0	S_1	0
2	0	S_1	S_2	0
3	1	S_2	S_3	0
4	1	S_3	S_1	**1**
5	0	S_1	S_2	0
6	1	S_2	S_3	0
7	1	S_3	S_1	**1**
8	0	S_1	S_2	0
9	0	S_2	S_0	0
10	1	S_0	S_1	0
11	0	S_1	S_2	0
12	1	S_2	S_3	0
13	0	S_3	S_2	0
14	1	S_2	S_3	0



Mealy: `det_out` is high *during* the clock in which the final 1 is on the input, so it is captured by the same edge that samples that bit. Clocks 4 and 7 show the overlap: "1011011" gives two hits.

RTL – 1011 overlapping, Mealy `seq1011_mealy.v`

In S3 the 1 that completes the pattern is also the start of the next one, so go to S1, not S0.

```
`timescale 1ns/1ps
// Overlapping "1011" detector -- Mealy FSM
module seq1011_mealy (
    input  wire clk,
    input  wire rst,
    input  wire serin,
    output reg  det_out
);
    localparam [1:0] S0 = 2'd0,   // idle
                     S1 = 2'd1,   // got "1"
                     S2 = 2'd2,   // got "10"
                     S3 = 2'd3;   // got "101"
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst)
        if (rst) state <= S0;
        else     state <= next_state;

```

```
    always @(*) begin
        next_state = S0;
        det_out    = 1'b0;
        case (state)
            S0: next_state = serin ? S1 : S0;
            S1: next_state = serin ? S1 : S2;   // "11" -> still "1"
            S2: next_state = serin ? S3 : S0;   // "100" -> nothing
            S3: if (serin) begin                // "1011" found!
                    next_state = S1;            // overlap: keep last "1"
                    det_out    = 1'b1;
                end else
                    next_state = S2;            // "1010" -> keep "10"
            default: next_state = S0;
        endcase
    end
endmodule
```

TB – Mealy output is checked just before the clock edge `tb_seq1011.v`

For a Mealy FSM, sample `det_out` at the rising edge with `$display` (old state, current input). Do not use `$strobe` here.

```
`timescale 1ns/1ps
module tb_seq1011;
    reg  clk = 1'b0, rst = 1'b1, serin = 1'b0;
    wire det_out;
    integer i, hits = 0;

    seq1011_mealy dut (.clk(clk), .rst(rst), .serin(serin), .det_out(det_out));

    always #5 clk = ~clk;

    initial begin
        $dumpfile("tb_seq1011.vcd");
        $dumpvars(0, tb_seq1011);
        repeat (2) @(posedge clk);
        #1 rst = 1'b0;
        for (i = 0; i < 100; i = i + 1) begin
            serin = $random;
            @(posedge clk); #1;
        end
        $display("DONE: %0d detections of 1011", hits);
        $finish;
    end

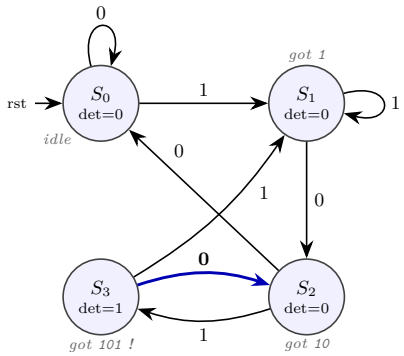
    always @(posedge clk)                   // values just BEFORE the edge
        if (!rst) begin
            $display("%6t  state=S%0d  serin=%b  det_out=%b",
                     $time, dut.state, serin, det_out);
            if (det_out) hits = hits + 1;
        end
endmodule
```

5 Predicted P4 – “101”: overlapping vs. non-overlapping (Moore)

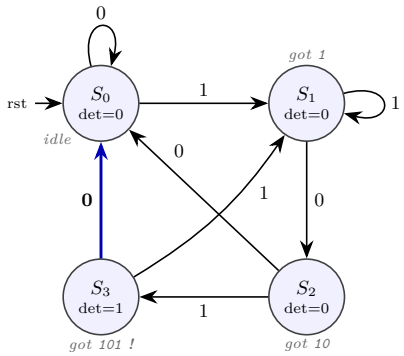
Likely wording

Design a Moore FSM to detect “101”. (a) overlapping, (b) non-overlapping. Compare the outputs for the same input stream.

(a) Overlapping

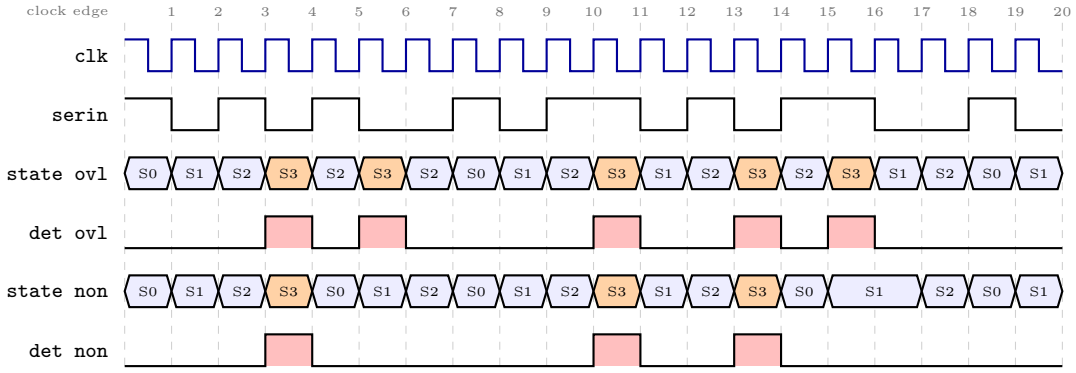


(b) Non-overlapping



The blue arrow is the only difference. With overlap, in S_3 (“101”) a 0 gives “1010”, whose ending “10” is reused, so go to S_2. Without overlap, the bits are used up, so go to S_0.

5.1 Waveform: same input, both machines



Input `10101` gives **two** hits with overlap (clocks 3 and 5) but only **one** without. That is exactly what the examiner wants you to show.

RTL – one parameter switches overlap on and off `seq101_moore.v`

Only the 0-branch out of S3 differs between the two versions.

```
`timescale 1ns/1ps
// "101" detector -- Moore FSM, OVERLAP = 1 (overlapping) or 0 (non-overlapping)
module seq101_moore #(parameter OVERLAP = 1) (
    input  wire clk,
    input  wire rst,
    input  wire serin,
    output reg  det_out
);
    localparam [1:0] S0 = 2'd0, S1 = 2'd1, S2 = 2'd2, S3 = 2'd3;
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst)
        if (rst) state <= S0;
        else     state <= next_state;

    always @(*) begin
        next_state = S0;
        case (state)
```

```
            S0: next_state = serin ? S1 : S0;
            S1: next_state = serin ? S1 : S2;
            S2: next_state = serin ? S3 : S0;
            S3: next_state = serin ? S1 : (OVERLAP ? S2 : S0);   // <-- the difference
            default: next_state = S0;
        endcase
    end

    always @(*) det_out = (state == S3);
endmodule
```

TB – both versions side by side on the same random input `tb_seq101.v`

```
`timescale 1ns/1ps
module tb_seq101;
    reg  clk = 1'b0, rst = 1'b1, serin = 1'b0;
    wire det_ov, det_nov;
    integer i;

    seq101_moore #(.OVERLAP(1)) u_ov  (.clk(clk), .rst(rst), .serin(serin), .det_out(det_ov));
    seq101_moore #(.OVERLAP(0)) u_nov (.clk(clk), .rst(rst), .serin(serin), .det_out(det_nov));

    always #5 clk = ~clk;

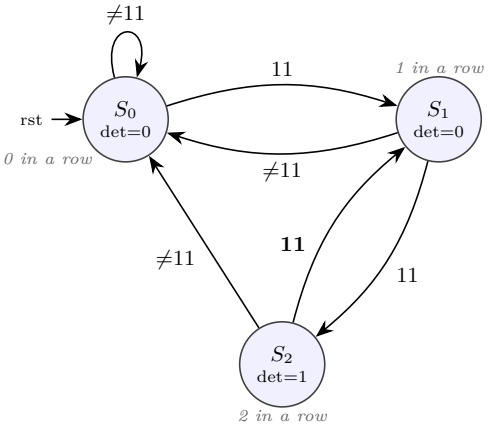
    initial begin
        $dumpfile("tb_seq101.vcd");
        $dumpvars(0, tb_seq101);
        repeat (2) @(posedge clk);
        #1 rst = 1'b0;
        for (i = 0; i < 100; i = i + 1) begin
            serin = $random;
            @(posedge clk); #1;
        end
        $finish;
    end

    always @(posedge clk)
        if (!rst)
            $strobe("%6t serin=%b | overlap: S%0d det=%b | non-overlap: S%0d det=%b",
                    $time, serin, u_ov.state, det_ov, u_nov.state, det_nov);
endmodule
```

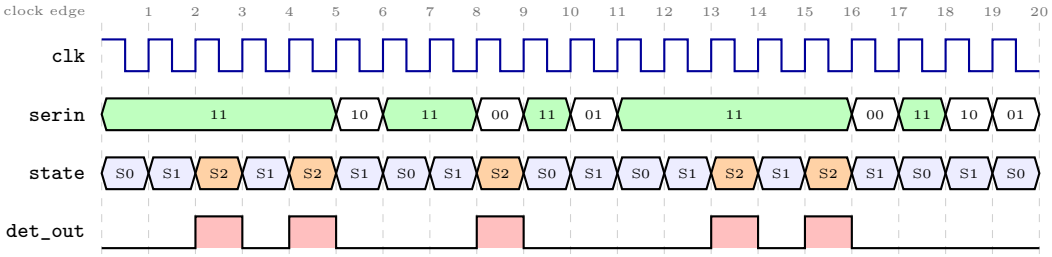
6 Predicted P5 – “11” twice in a row on a 2-bit input (non-overlapping)

Likely wording (a Q1 twin)

Assert `det_out` when “11” is received on a 2-bit input for two consecutive clocks, *non-overlapping*. Verify with 100 random clocks.



After a detection (S_2) another 11 is the *first* of a new pair, so go to S_1, not S_2. So “11 11 11 11” gives hits after the 2nd and 4th values only.



RTL – 2-bit input, 11 on 2 consecutive clocks, non-overlapping `det11x2.v`

This is the same counter-chain idea as Q1 with fewer states. Non-overlap means that after a hit, a new 11 counts as the first one again.

```
`timescale 1ns/1ps
// det_out = 1 when serin == 2'b11 on 2 consecutive clocks, NON-overlapping
module det11x2 (
    input  wire       clk,
    input  wire       rst,
    input  wire [1:0] serin,
    output reg        det_out
);
    localparam [1:0] S0 = 2'd0,   // 0 in a row
                     S1 = 2'd1,   // 1 in a row
                     S2 = 2'd2;   // 2 in a row -> det_out = 1
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst)
        if (rst) state <= S0;
        else     state <= next_state;

    always @(*) begin
        next_state = S0;
        if (serin == 2'b11)
            case (state)
                S0:      next_state = S1;
                S1:      next_state = S2;
                S2:      next_state = S1;   // non-overlap: count restarts at 1
                default: next_state = S0;   // (overlap would be: S2)
            endcase
```

```
    end

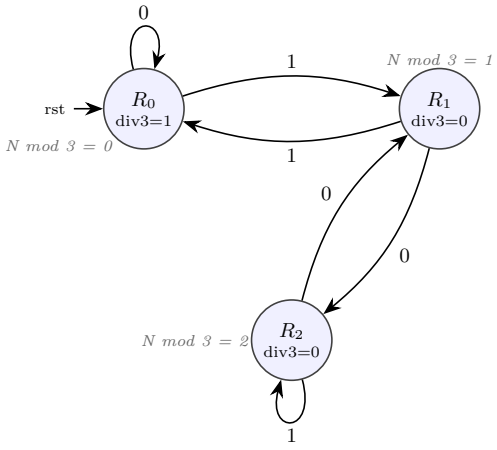
    always @(*) det_out = (state == S2);
endmodule
```

Testbench: copy `tb_det01x3.v`, change the module name and compare against `2'b11`. The repository has the RTL ready.

7 Predicted P6 – serial number divisible by 3

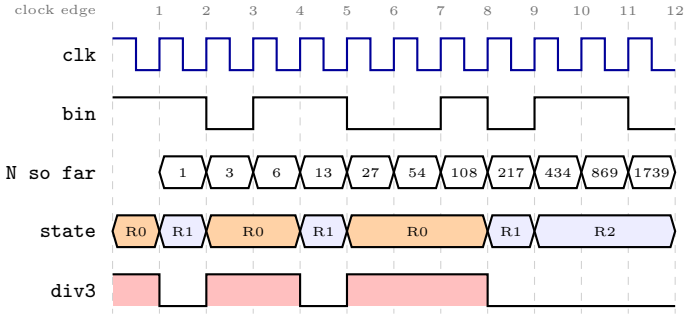
Likely wording

A binary number arrives serially, MSB first. Assert `div3` whenever the number received so far is divisible by 3.



r	bin=0 ($2r$)	bin=1 ($2r+1$)
0	0	1
1	2	0
2	1	2

The state stores the remainder, not the number, so three states are enough for any length. Generalise: divisible by m needs m states, next $= (2r + b) \bmod m$.



RTL – remainder FSM (binary number arrives MSB first) `div3_fsm.v`

Shifting in bit b turns N into $2N + b$, so the next remainder is $(2r + b) \bmod 3$. Three states, one per remainder.

```
`timescale 1ns/1ps
// div3 = 1 when the binary number received so far (MSB first) is divisible by 3
module div3_fsm (
    input  wire clk,
    input  wire rst,
    input  wire bin,       // serial bit, MSB first
    output reg  div3
);
    localparam [1:0] R0 = 2'd0, R1 = 2'd1, R2 = 2'd2;   // remainder 0/1/2
    reg [1:0] state, next_state;

    always @(posedge clk or posedge rst)
        if (rst) state <= R0;
        else     state <= next_state;

    always @(*) begin
        case (state)                      // next = (2*r + bin) mod 3
            R0:      next_state = bin ? R1 : R0;
            R1:      next_state = bin ? R0 : R2;
            R2:      next_state = bin ? R2 : R1;
```

```
            default: next_state = R0;
        endcase
    end

    always @(*) div3 = (state == R0);
endmodule
```

8 Quick variations – state tables you can draw in 2 minutes

"110" non-overlapping, Mealy

	in=0	in=1
S_0	$S_0/0$	$S_1/0$
S_1 (1)	$S_0/0$	$S_2/0$
S_2 (11)	$S_0/\mathbf{1}$	$S_2/0$

"111" still ends in "11" so stay in S_2.

"1001" overlapping, Mealy

	in=0	in=1
S_0	$S_0/0$	$S_1/0$
S_1 (1)	$S_2/0$	$S_1/0$
S_2 (10)	$S_3/0$	$S_1/0$
S_3 (100)	$S_0/0$	$S_1/\mathbf{1}$

After "1001" the last 1 is reused, so go to S_1.

Three consecutive 1s ("111"), overlapping, Moore

	in=0	in=1	out
S_0	S_0	S_1	0
S_1	S_0	S_2	0
S_2	S_0	S_3	0
S_3	S_0	S_3	**1**

"0110" overlapping, Moore

	in=0	in=1	out
S_0	S_1	S_0	0
S_1 (0)	S_1	S_2	0
S_2 (01)	S_1	S_3	0
S_3 (011)	S_4	S_0	0
S_4 (0110)	S_1	S_2	**1**

From S_4: "01100" ends in "0", and "01101" ends in "01".

Even number of 1s (parity), Moore

	in=0	in=1	out
EVEN	EVEN	ODD	1
ODD	ODD	EVEN	0

Converting Moore → Mealy: move each state's output onto every arrow *entering* it, then merge states that now behave identically ("010": 4 → 3 states).

Mealy → Moore: split any state that is entered with different outputs.

Likely short (theory) questions

1. Differentiate Moore and Mealy FSMs. Which one needs more states, and which responds faster?
2. What is the difference between overlapping and non-overlapping detection? Show it with a waveform.
3. Why are FSMs coded with separate sequential and combinational always blocks?
4. What causes an unintended latch in an FSM, and how do you avoid it?
5. Binary vs. one-hot vs. Gray state encoding: how many flip-flops does each need for N states? (binary $\lceil \log_2 N \rceil$, one-hot N, Gray $\lceil \log_2 N \rceil$.)
6. Why does the testbench change inputs on the negative edge or after a small delay, and not at `posedge`?
7. Synchronous vs. asynchronous reset: write both forms of the state register.

Synchronous vs. asynchronous reset snippet

```
always @(posedge clk or posedge rst)   // ASYNC: reset acts immediately
    if (rst) state <= S0; else state <= next_state;

always @(posedge clk)                  // SYNC: reset only at a clock edge
    if (rst) state <= S0; else state <= next_state;
```

Running the code

`iverilog -o sim rtl/seq010_moore.v tb/tb_seq010.v && vvp sim && gtkwave tb_seq010.vcd`
The same works for every `rtl/*.v` + `tb/tb_*.v` pair. Signals to add in GTKWave: `clk, rst, serin, dut.state, det_out`.